

Movie Recommendation System Report

1. Introduction

Movie recommendation systems are crucial resources for assisting consumers in finding information they are likely to find entertaining. By sifting through vast movie libraries and suggesting only those that suit the user's preferences, they customize the user experience.

In this assignment, we implemented three different approaches for building a movie recommendation system:

- **User-Based Collaborative Filtering:** Recommends movies based on the preferences of similar users.
- **Item-Based Collaborative Filtering:** Recommends movies similar to those a user has already liked.
- **Random-Walk-Based (Pixie-Inspired):** Uses graph traversal techniques to explore connections between users and movies.

2. Dataset Description

We used the **MovieLens 100K dataset**, which contains:

- 943 users
- 1,682 movies
- 100,000 ratings

Each rating record includes:

- `user_id`
- `movie_id`
- `rating` (on a scale of 1 to 5)

- `timestamp`

Additional files provided:

- `u.item`: movie metadata (title, release date)
- `u.user`: user demographic info (age, gender, occupation)

Preprocessing:

- Renamed columns for readability.
 - Converted Unix timestamps to human-readable dates.
 - Removed duplicates and handled any missing values.
 - Normalized user ratings for the Pixie-inspired method.
 - Exported cleaned DataFrames to `ratings.csv`, `movies.csv`, and `users.csv`
-

3. Methodology

a. User-Based Collaborative Filtering

- Built a **user-movie rating matrix**.
- Used **cosine similarity** to compare users based on common rated items.
- Identified top-N similar users and used their ratings to predict movies for the target user.

b. Item-Based Collaborative Filtering

- Transposed the rating matrix to focus on movie-to-movie similarity.
- Used **cosine similarity** to identify movies that are often rated similarly by the same users.

- Recommended top-N similar movies to a given movie.

c. Pixie-Inspired Graph-Based Recommendation

- Modeled the dataset as a **bipartite graph**:
 - Nodes: users and movies
 - Edges: a rating interaction
 - Normalized user ratings to center around zero.
 - Performed **random walks** from a starting node (user or movie) to explore neighboring movies.
 - Counted movie visits during the walk to determine recommendation ranking.
-

4. Implementation Details

Function Development

- Used **pandas** for data handling and transformation.
- Built functions for each recommendation technique with reusable structure and clear inputs.

Graph Creation

To model the user–movie interactions as a **bipartite graph**, we used an **adjacency list** representation. In this graph:

- **Users** and **movies** are nodes.
- An **edge** connects a user to a movie if the user has rated that movie.

This structure allows us to easily navigate between related users and movies, enabling random walk–based recommendations.

Random Walk Logic

- For each walk step:
 - Choose a random neighbor of the current node.
 - If the node is a movie, increase its visit count.
- After `walk_length` steps, sort movies by frequency of visits and return the top-N.

5. Results and Evaluation

Sample Outputs

User-Based:

Recommendations for user `10`:

Movie Name	
Ranking	
1	GoldenEye (1995)
2	Four Rooms (1995)
3	Copycat (1995)
4	Shanghai Triad (Yao a yao yao dao waipo qiao) ...
5	Babe (1995)

Item-Based:

Recommendations for *Jurassic Park* (1993):

movie_name	
ranking	
1	Top Gun (1986)
2	Speed (1994)
3	Raiders of the Lost Ark (1981)
4	Empire Strikes Back, The (1980)
5	Indiana Jones and the Last Crusade (1989)

Pixie-Inspired (User 1):

		Movie Name
Ranking		
1		Aristocats, The (1970)
2		Twilight (1998)
3	Willy Wonka and the Chocolate Factory	(1971)
4		Go Fish (1994)
5		Twelve Monkeys (1995)

Comparison:

Method	Strengths	Limitations
User-Based	Personalized, simple to implement	Struggles with sparse data
Item-Based	Good for finding similar movies	Ignores user preferences
Pixie-Inspired	Fast, flexible, explores local neighborhoods	Randomness can affect consistency

Limitations in the Implementation:

- **Cold Start Problem:** User/item based recommendation need users or movies to work with, otherwise there cannot be any recommendation on a new user/movies.
- **Data Sparsity:** Most of these users rated very few movies, which can make collaborative filtering less effective in calculating similarity.
- **No Implicit Feedback:** The system relies solely on explicit ratings and overlooks other signals such as watch time, click-throughs, or genres of the movies
- **Random Walk Variability:** In the case of the Pixie model, each time you are running the program due to random walk (especially in case of short walk length) you still could

have small difference in the final results.

- **No Evaluation Metric:** Ground truth is missing so we could not calculate the metrics such as RMSE, Precision@k, Recall, etc.

6. Conclusion

This assignment focuses on three different movie recommendation algorithms, comprising user-based collaborative filtering, item-based filtering, and a Pixie algorithm building initiative. Using each method showed strengths—user-based filtering captured personalization, item-based filtering provided consistency, and graph-based walks uncovered latent structures. It managed data preparation, similarity calculation, and random-walk access in a bipartite user-movie graph. Although the models showed satisfactory performance, drawbacks like sparsity of data, absence of implicit feedback, and cold-start problem were found. Acquisition of new techniques, such as hybrid and genre combinations, and evaluation methods — such as Precision@K. In conclusion, this project demonstrated the utility and versatility of recommendation systems in practice.